Лабораторная работа 6. Мультиагентная система: оркестратор и специализированные исполнители

Главный вопрос. Когда разделение работы между специализированными агентами окупает дополнительные вызовы модели?
Что получится. Мультиагентный вариант агента из ЛР4: оркестратор делит задачу на подзадачи и поручает их трём исполнителям — для почты, календаря и файлов, у каждого свои инструменты и свой контекст. Сравнение с одиночным агентом по успешности и затратам на 30 задачах AgentDojo, разбор передачи информации между агентами; отчёт.
Среда. Обвязка ЛР4 и оценочный контур ЛР5, AgentDojo (`workspace`), сервер модели из ЛР1 или внешний API.

1. Зачем эта работа

Одиночный агент из ЛР4 в каждом вызове видит все 24 инструмента и всю историю эпизода. На задаче «Please send a summary of the file 'client-meeting-minutes.docx' to the client via email» в его контекст попадают описания инструментов календаря, которые не нужны, полный текст файла и всё, что агент успел прочитать. Маленькой модели трудно выбрать нужный инструмент из длинного списка, а длинный контекст дорог и мешает сосредоточиться.

Мультиагентная система делит работу. **Оркестратор** разбирает задачу на подзадачи и поручает их **исполнителям**. Исполнитель «почты» знает только инструменты почты и контактов, исполнитель «файлов» — только инструменты диска. Каждый начинает с чистого контекста и возвращает оркестратору короткий итог. Такой шаблон называют orchestrator-workers [1] или «агенты как инструменты» [2].

У разделения есть цена. Вызовов модели становится больше: по оценке Anthropic, агент тратит примерно в 4 раза больше токенов, чем обычный чат, а мультиагентная система — примерно в 15 раз [3]. Там же сказано, где разделение окупается: когда подзадачи независимы, а нужные сведения не помещаются в один контекст, — и где нет: когда агентам нужен общий контекст. Информация теряется при передаче: исполнитель вернёт краткий пересказ, а оркестратору нужна была точная дата. Решения, принятые разными агентами без общего контекста, могут противоречить друг другу, — поэтому есть и прямые рекомендации не строить мультиагентные системы там, где хватает одного агента [4]. Проверьте, где правы те и другие, на своих задачах.

2. Как устроено

2.1. Роли

Роль	Что видит	Инструменты
Оркестратор	Запрос пользователя, свою историю, итоги исполнителей	`ask_email_agent(task)`, `ask_calendar_agent(task)`, `ask_files_agent(task)`
Исполнитель «почта»	Только текст подзадачи от оркестратора	7 инструментов почты и 2 поиска контактов
Исполнитель «календарь»	Только текст подзадачи	7 инструментов календаря
Исполнитель «файлы»	Только текст подзадачи	8 инструментов диска

Связь между агентами здесь строго иерархическая, в отличие от систем, где агенты свободно переписываются друг с другом [6]. Оркестратор сам не касается среды: всё делают исполнители. Для оркестратора исполнитель — обычный инструмент: оркестратор вызывает его с текстом подзадачи и получает текст итога. Внутри вызова работает цикл ReAct из ЛР4 со своими сообщениями

и своим подмножеством инструментов. Исполнители работают с общей средой: письмо, отправленное «почтой», сразу видно всем.

Рис. 1 показывает устройство.

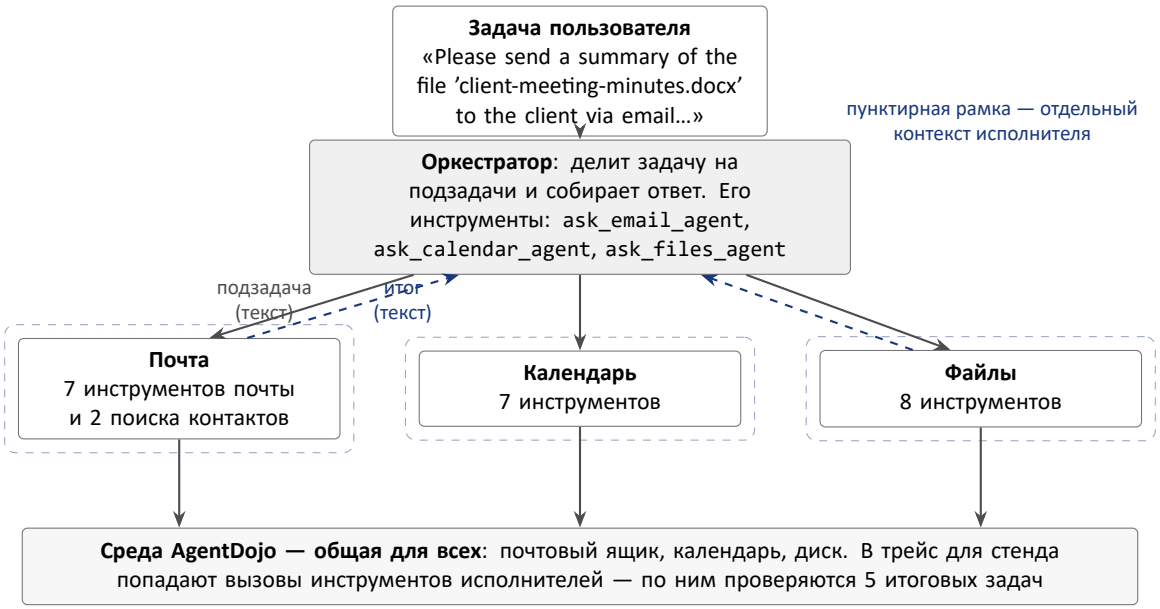


Рис. 1. Оркестратор и три исполнителя с отдельными контекстами над общей средой

2.2. Лимиты и сопоставимость

Чтобы сравнение с одиночным агентом было честным, общий лимит вызовов **инструментов среды** тот же, что в ЛР4, — 15 на эпизод, счётчик общий для всех исполнителей. Кроме того: оркестратор — до 8 вызовов модели, каждый исполнитель — до 8 вызовов на одно поручение, всего не больше 30 вызовов модели на эпизод. Модель, температура и лимит выходных токенов — те же, что в ЛР4. Число вызовов модели при этом будет разным, и его нужно измерить: это и есть цена разделения.

2.3. Трейс для стенда

У пяти итоговых задач (19, 23, 37, 38, 39) стенд проверяет последовательность вызовов инструментов. Поэтому в сообщения, которые агент возвращает стенду, записываются вызовы инструментов исполнителей в том порядке, в каком они выполнялись. Вызовы-поручения `ask_*_agent` туда **не** записываются: для стенда это не инструменты среды, и лишние вызовы в последовательности сломают проверку. Полная картина — кто кого и о чём попросил — сохраняется в вашем собственном трейсе (ЛР4, п. 2.3): у каждого шага есть роль и номер поручения.

2.4. Группы задач

Из 30 итоговых задач 7 затрагивают два или три сервиса (номера 13, 15, 18, 19, 23, 25, 33), остальные 23 — один. Если разделение работы полезно, это прежде всего должно проявиться на первой группе. Вторая показывает цену лишней координации там, где хватило бы одного исполнителя.

2.5. Метрики

- **Успешность** по проверке стенда — всего и по двум группам.
- **Затраты**: вызовы модели по ролям, входные и выходные токены, время эпизода.
- **Нагрузка на контекст** — наибольшее число входных токенов в одном вызове модели за эпизод. Разделение должно её снижать.
- **Причины неудач** в мультиагентном варианте: оркестратор поручил не тому исполнителю или не всё; исполнитель ошибся с инструментом или аргументами; нужные сведения потерялись в итоге исполнителя; оркестратор неверно собрал ответ.

3. Что сделать

3.1. Мультиагентный вариант

1. Оркестратор: системный промпт (роль, какие исполнители есть и что они умеют, правила: давать исполнителю самодостаточную подзадачу со всеми нужными именами, датами и адресами, в конце ответить пользователю), три инструмента-поручения с описанием и одним строковым параметром `task`.
2. Исполнители: цикл ReAct из ЛР4 со своим системным промптом, подмножеством инструментов и лимитами из п. 2.2. Итог исполнителя — его последнее текстовое сообщение; в промпте попросите включать в итог конкретные данные: адреса, даты, идентификаторы файлов.
3. Сообщения для стенда и собственный трейс — по п. 2.3.
4. Переключение архитектуры — параметром конфигурации; одиночный агент — обвязка ЛР4 без изменений.

3.2. Проверки без модели

- Сценарный оркестратор, который вызывает «файлы», а затем «почту» с заранее записанными поручениями, приводит к тем же вызовам инструментов, что и эталон задачи 33, и стенд засчитывает задачу.
- Общий счётчик инструментов останавливает эпизод на 15-м вызове, даже если вызовы распределены между исполнителями.
- В сообщениях для стенда нет вызовов `ask_*_agent`, а порядок вызовов инструментов совпадает с порядком выполнения.

4. Эксперименты

До запусков запишите прогноз: на какой группе задач мультиагентный вариант выиграет, на какой проиграет; во сколько раз вырастет число вызовов модели; снизится ли нагрузка на контекст.

4.1. Отладка

На 10 отладочных задачах добейтесь работы оркестратора и исполнителей, изучите трейсы и поправьте промпты. Затем зафиксируйте конфигурацию.

4.2. Эксперимент 1. Один агент или команда

Запустите мультиагентный вариант на 30 итоговых задачах при температуре 0, по одному эпизоду. Одиночный агент — первый прогон при температуре 0 из ЛР5, если с тех пор не менялись модель, промпты и лимиты; иначе запустите его заново.

1. Успешность обоих вариантов, всего и по группам из п. 2.4; попарные переходы («Правила», п. 6). В группе из 7 задач одна задача — 14 п.п., поэтому пишите выводы в штуках.
2. Затраты: медианы и диапазоны числа вызовов модели, токенов и времени; отношение затрат мультиагентного варианта к одиночному.
3. Нагрузка на контекст: медиана и максимум наибольшего входа в одном вызове для обоих вариантов.

4.3. Эксперимент 2. Что видит исполнитель

Возьмите 7 многосервисных задач и 3 односервисные, на которых мультиагентный вариант ошибся или потратил больше всего токенов. Повторите их в варианте, где исполнитель получает не только подзадачу, но и полный исходный запрос пользователя. Изменились ли успешность и число потерь информации? Как изменилась нагрузка на контекст?

4.4. Разбор передачи информации

Разберите все неудачи мультиагентного варианта на многосервисных задачах и две неудачи на односервисных; если неудач мало, разберите и успешные эпизоды с наибольшими затратами. Для каждого эпизода покажите поручения оркестратора и итоги исполнителей и назовите причину

по списку из п. 2.5. Где именно потерялось нужное — в формулировке поручения, в работе исполнителя, в его итоге или при сборке ответа?

4.5. По желанию

- Соберите тот же вариант на готовом фреймворке — супервизор в LangGraph [5] или «агенты как инструменты» в OpenAI Agents SDK [2] — и сравните объём кода и удобство отладки.
- Разрешите оркестратору поручать независимые подзадачи параллельно и сравните время эпизода.
- Подумайте о безопасности: исполнитель «файлы» не может отправлять письма. Какую атаку из ЛР8 это остановило бы и какую нет?

5. Что сдать

- Код оркестратора и исполнителей, их промпты, конфигурации обоих вариантов.
- Трейсы, результаты серий и автоматически построенные таблицы.
- Отчёт на 3–4 страницы: прогноз; успешность и переходы по группам; затраты и нагрузка на контекст; эксперимент 2; разбор передачи информации; вывод — когда разделение окупается; кодогенерация.

Отсутствие выигрыша — допустимый и содержательный результат. Ошибки постановки здесь чаще всего такие: разные лимиты инструментов в сравниваемых вариантах; в сообщения для стенда попали вызовы-поручения; промпты правились на итоговых задачах.

Вопросы к защите

1. Покажите поручение оркестратора, после которого исполнитель не смог выполнить подзадачу. Что в нём было недостаточно?
2. Почему вызовы `ask_*_agent` нельзя записывать в сообщения для стенда? Что сломается в задаче 19?
3. Во сколько раз больше вызовов модели делает мультиагентный вариант? Откуда берётся эта разница на вашем примере?
4. Нагрузка на контекст снизилась, а успешность нет. Как это объяснить?
5. Какие задачи выиграли от передачи исполнителю полного запроса и чем за это пришлось заплатить?
6. Дайте исполнителю «почта» ещё и инструменты календаря. Предскажите, как изменится поведение оркестратора на задаче 15, и проверьте.
7. Для каких задач вашего бюджета и вашей модели вы выбрали бы один агент, а для каких — команду?

Источники

1. Anthropic. Building effective agents. 2024 — шаблон orchestrator-workers и когда он оправдан.
2. OpenAI Agents SDK. Orchestrating multiple agents — «агенты как инструменты», когда управление остаётся у оркестратора, и передача управления (handoffs), когда оно переходит к исполнителю.
3. Anthropic. How we built our multi-agent research system. 2025 — когда мультиагентность окупается и сколько токенов она стоит.
4. Cognition. Don’t Build Multi-Agents. 2025 — аргументы против: потеря общего контекста и противоречивые решения.
5. LangChain. Multi-agent — схемы мультиагентных систем, в том числе супервизор с исполнителями-инструментами.
6. Wu Q. et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. 2023. Статья — агенты, общающиеся сообщениями (разд. 2).

Приложение. Сообщения для стенда

Исполнитель выполняет вызовы через тот же `runtime` и ту же среду, что получил оркестратор от стенда, и дописывает в общий список сообщений для стенда пары «вызов — результат» (формат — ЛР4, приложение):

```
def run_worker(role, task, runtime, env, stand_log, counter):
    tools = [to_openai_tool(runtime.functions[name]) for name in WORKER_TOOLS[role]]
    chat = [{"role": "system", "content": WORKER_PROMPTS[role]}, {"role": "user", "content": task}]
    # ... цикл ReAct из ЛР4; для каждого вызова инструмента среды:
    #     counter.tools += 1                    # общий лимит — 15 на эпизод
    #     result, error = runtime.run_function(env, name, args)
    #     stand_log += [ChatAssistantMessage(...tool_calls=[call]), ChatToolResultMessage(...)]
    return final_text                           # итог исполнителя — строка для оркестратора
```

Оркестратор обрабатывает вызов `ask_email_agent` так: запускает `run_worker("email", args["task"], …)` и возвращает строку итога как результат инструмента в **своей** истории. В `stand_log` этот вызов не попадает. В конце эпизода в `stand_log` добавляется финальный ответ оркестратора текстом.